UNIT KOMPETENSI 09 • TI.SQL.09.2026

MENGGUNAKAN SQL

Implementasi Riil pada Proyek: Portal Karir, Profil Industri & Modul Automation-GPT (Fullstack Web)

Presenter / Mahasiswa	Aditya Ristanto Mukti (Fullstack Web Developer Intern)	**Repository**	`github.com/AdityaRistantoMukti/Website_Intersis`
Institusi / Mitra	PT Global Philsonic Technologies (Intersis) Batam	**Tech Stack Utama**	React 19, Vite, Tailwind CSS v4, Node.js Express 5, MySQL2
Cakupan Outline Sub-Title	Operasi DML • Manipulasi tabel/view • Stored procedure • Function • Trigger • Commit/Rollback		

I. PETA OUTLINE & STRUKTUR PEMBAHASAN

Unit kompetensi **Menggunakan SQL** ini disusun secara berkesinambungan mencakup seluruh sub-judul yang didefinisikan pada diagram alur kompetensi, diaplikasikan langsung pada arsitektur website yang telah dibangun:

- **Operasi DML**: Mengeksekusi manipulasi data (SELECT, INSERT, UPDATE, DELETE) menggunakan Prepared Statements berparameter aman.
- **Manipulasi tabel/view**: Mengelola struktur skema tabel melalui DDL (ALTER TABLE, migrasi bilingual, produk) dan pembuatan virtual view pelaporan.
- **Stored procedure**: Menyusun prosedur tersimpan di database engine untuk eksekusi batch kompleks seperti otomasi penutupan lowongan kadaluarsa.
- **Function**: Memanfaatkan fungsi agregasi bawaan (COUNT, NOW, COALESCE) dan User Defined Functions untuk komputasi data terpusat.
- **Trigger**: Mengimplementasikan pemicu database otomatis (AFTER INSERT/UPDATE) untuk mencatat histori perubahan ke activity_logs.
- **Commit/Rollback**: Menerapkan prinsip ACID transaksi multi-tabel di Node.js (beginTransaction, commit, rollback) untuk menjaga integritas data.

II. NASKAH WICARA LENGKAP PRESENTER (VERBATIM SPEECH SCRIPT)

🎤 PANDUAN NARASI WICARA PRESENTER (SPEAKING SCRIPT)

[Pembukaan & Sikap] "Selamat pagi/siang Dewan Penguji yang saya hormati. Structured Query Language (SQL) merupakan instrumen inti dalam interaksi data pada sistem berbasis relasional. Pada unit kompetensi **Menggunakan SQL**, saya akan mendemonstrasikan penguasaan menyeluruh dari operasi DML tingkat lanjut hingga manajemen transaksi perbankan/enterprise (ACID)."

[Menjelaskan Sub 1 & 2: Operasi DML dan Manipulasi Tabel/View] "Pertama, dalam *Operasi DML (Data Manipulation Language)*, kami menulis perintah `SELECT`, `INSERT`, `UPDATE`, dan `DELETE` dengan standar keamanan tertinggi, yaitu selalu menggunakan **Prepared Statements** bertanda tanya (`?`). Pendekatan ini secara mutlak memblokir potensi serangan SQL Injection. Terkait *Manipulasi Tabel/View*, kami menyusun skrip migrasi DDL terstruktur seperti `add_bilingual_support.sql` dan `add_products_table.sql` untuk memperluas skema tabel tanpa merusak integritas data eksisting."

[Menjelaskan Sub 3, 4, & 5: Stored Procedure, Function, dan Trigger] "Kedua, kami mendalami otomasi tingkat basis data melalui *Stored Procedure, Function, dan Trigger*. Kami merancang Stored Procedure `sp_archive_expired_jobs()` yang dapat dipanggil secara terjadwal untuk mengubah status lowongan yang melewati batas waktu menjadi 'closed' secara otomatis. Kami memanfaatkan fungsi SQL bawaan seperti `COUNT()`, `NOW()`, dan `COALESCE()` untuk agregasi ringkasan dashboard, serta merancang konsep **Database Trigger** (`trg_after_job_update`) yang secara otomatis menyisipkan rekaman log ke tabel `activity_logs` setiap kali ada mutasi status lowongan kerja."

[Menjelaskan Sub 6: Commit/Rollback (Transaksi ACID)] "Ketiga, pilar paling kritikal adalah *Commit & Rollback*. Dalam transaksi multi-langkah—seperti saat memperbarui lowongan pekerjaan sekaligus mencatat entri log audit—kami membungkus operasi tersebut di dalam mekanisme transaksi database MySQL. Kami menjalankan `await connection.beginTransaction()`. Jika seluruh proses berhasil, kami mengeksekusi `commit()`. Namun jika salah satu tahapan gagal, seluruh perubahan seketika kami batalkan melalui `rollback()`. Dengan cara ini, database kami selalu konsisten dan terlindungi dari kondisi data korup."

III. PEMBAHASAN TEKNIS RINCI & BEDAH KODE NYATA PROYEK

Berikut adalah pembuktian implementasi teknis dari masing-masing sub-judul berdasarkan baris kode dan berkas nyata di dalam repository proyek:

9.1 & 9.2. Operasi DML Aman (Prepared Statements) & Migrasi Skema DDL

Kaitkan ke Codebase: **`backend/src/controllers/admin/adminJobVacancyController.js & database/add_products_table.sql`**

Operasi DML selalu menggunakan prepared statement binding untuk memastikan parameter masukan tidak diinterpretasikan sebagai kode SQL:

backend/src/controllers/admin/adminJobVacancyController.js

```
// Operasi DML Terparameterisasi (Pencegahan SQL Injection)
exports.createJob = async (req, res, next) => {
  try {
    const { category_id, title, location, employment_type, description, requirements, form_url, status } = req.body;
    const adminId = req.user.id;

    const query = `
      INSERT INTO job_vacancies
      (category_id, title, location, employment_type, description, requirements, form_url, status, created_by)
      VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    `;
    const [result] = await db.query(query, [
      category_id, title, location, employment_type, description, requirements, form_url, status || 'draft', adminId
    ]);

    res.status(201).json({ success: true, message: 'Lowongan berhasil dipublikasikan.', id: result.insertId });
  } catch (err) {
    next(err);
  }
};
```

9.3, 9.4 & 9.5. Penerapan Stored Procedure, Built-in Functions & Trigger

Kaitkan ke Codebase: **`database/isa_career_hrd.sql (Prosedur & Trigger)`**

Pemanfaatan fitur native engine MySQL InnoDB untuk automasi pemeliharaan data dan jejak audit:

database/stored_procedures_triggers.sql

```
-- 1. Stored Procedure: Otomasi Penutupan Lowongan Kadaluarsa
DELIMITER //
CREATE PROCEDURE sp_archive_expired_jobs(IN p_days_threshold INT)
BEGIN
  UPDATE job_vacancies
  SET status = 'closed', updated_at = NOW()
  WHERE status = 'published'
    AND DATEDIFF(NOW(), created_at) > p_days_threshold;
END //
DELIMITER ;

-- 2. Trigger Otomatis: Pencatatan Audit Trail Pasca Mutasi Lowongan
DELIMITER //
CREATE TRIGGER trg_audit_job_update
AFTER UPDATE ON job_vacancies
FOR EACH ROW
BEGIN
  IF OLD.status <> NEW.status THEN
    INSERT INTO activity_logs (user_id, action, module, description, created_at)
    VALUES (NEW.created_by, 'UPDATE_STATUS', 'CAREER',
            CONCAT('Status lowongan "', NEW.title, '" diubah dari ', OLD.status, ' ke ', NEW.status), NOW());
  END IF;
END //
DELIMITER ;
```

9.6. Manajemen Transaksi ACID (Begin Transaction, Commit, Rollback)

Kaitkan ke Codebase: **`backend/src/config/database.js & Contoh Transaksi Controller`**

Prinsip Atomicity diimplementasikan pada operasi yang melibatkan lebih dari satu tabel secara simultan:

backend/src/utils/transactionDemo.js

```
// Implementasi Transaksi ACID: Menjamin Operasi All-or-Nothing
const db = require('../config/database');

async function executeTransactionalJobCreation(jobData, adminId) {
  // 1. Dapatkan dedicated connection dari connection pool
  const connection = await db.getConnection();
  try {
    // 2. Mulai transaksi
    await connection.beginTransaction();

    // Langkah A: Insert lowongan kerja
    const [jobResult] = await connection.query(
      'INSERT INTO job_vacancies (category_id, title, description, requirements, created_by) VALUES (?, ?, ?, ?, ?)',
      [jobData.category_id, jobData.title, jobData.description, jobData.requirements, adminId]
    );

    // Langkah B: Insert activity audit log
    await connection.query(
      'INSERT INTO activity_logs (user_id, action, module, description) VALUES (?, "CREATE", "CAREER", ?)',
      [adminId, `Menerbitkan lowongan baru ID: ${jobResult.insertId}`]
    );

    // 3. Jika kedua query sukses, komit perubahan permanen
    await connection.commit();
    return { success: true, jobId: jobResult.insertId };
  } catch (error) {
    // 4. Jika ada tahapan yang gagal, batalkan seluruh mutasi data
    await connection.rollback();
    throw error;
  } finally {
    // 5. Kembalikan koneksi ke pool
    connection.release();
  }
}
```

IV. POIN KUNCI PRESENTASI (SLIDE TALKING POINTS)

POINTER RINGKAS UNTUK TAMPILAN SLIDE / LAYAR

- DML Berparameter: Prepared statement placeholder '?' untuk mengeliminasi celah SQL Injection.
- Manipulasi Skema DDL: Skrip migrasi terstruktur untuk ekspansi tabel produk dan bilingual.
- Stored Procedure: Prosedur pemeliharaan berkala penutupan lowongan kadaluarsa.
- Database Trigger: Logging audit trail instan berbasis event INSERT/UPDATE di level database.
- Transaksi ACID Penuh: Menjamin integritas data all-or-nothing dengan beginTransaction, commit, dan rollback.

V. MATRIKS TANYA JAWAB & PANDUAN PEMBELAAN SIDANG (Q&A DEFENSE)

Prediksi pertanyaan evaluasi dari Penguji / Asesor / Pembimbing Industri beserta jawaban teknis yang siap dipertahankan:

Q1: Mengapa Anda memilih menggunakan Prepared Statements di library mysql2 dibandingkan string concatenation biasa?

Jawaban Pembelaan Presenter: Prepared statements memisahkan antara struktur query SQL dan nilai data yang dikirimkan. Database engine mengompilasi struktur query terlebih dahulu sebelum menyisipkan data parameter, sehingga karakter khusus dari input pengguna (seperti tanda petik atau statement OR 1=1) diperlakukan murni sebagai literal string, bukan perintah SQL yang dapat dieksekusi (kebal SQL Injection).

Q2: Dalam situasi apa saja kita wajib menggunakan mekanisme Commit dan Rollback?

Jawaban Pembelaan Presenter: Transaksi commit dan rollback wajib digunakan pada setiap proses bisnis yang melibatkan lebih dari satu operasi penulisan data yang saling bergantung (multi-table write). Misalnya: saat memotong saldo pengguna sekaligus membuat riwayat tagihan, atau saat merilis lowongan kerja sekaligus mencatat log audit ke sistem HRD.